PROJECT-BASED LEARNING

PHASE-II EVALUATION

EcoRouting Engine: Hazard-Aware Path Optimization and Fill-Rate-Adaptive Waste Scheduling

Team ID: DSCPP-III-2026-T186

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T186
Semester: 3rd
Section: A / D / H
Domain: Graph Algorithms (C++)

Team Members

1. Tejas Jain – 2510011541 (Team Lead)
2. Apoorv Negi – 2510011088
3. Yash Nigam – 2510010441

Mentor

Mr. Navneet Rajput

Interactions completed: _____

Phase-I Recap & Feedback Incorporated

Phase-I Status

In Phase-I we defined the problem statement, planned the system architecture, chose the tech stack, and picked a modified (zone-constrained) Dijkstra for hazardous waste and a fill-rate formula for scheduling. All of that is now built and working.

Feedback Received

- Fill-rate-based scheduling
- Public-safety-first route planning for hazardous waste
- En-route pickups
- Limit on truck capacity and number of trucks

Changes Incorporated

- Home due once capacity / fill rate days pass (1–14); most overdue first
- Hazard trucks use constrained Dijkstra on industrial and arterial roads only
- En-route pickup: later stops on the path are emptied on the way, no backtracking
- Default sim settings: 3 normal + 2 hazard trucks, 1000 kg per truck; overflow waits a day

Problem Statement & Finalized Objectives

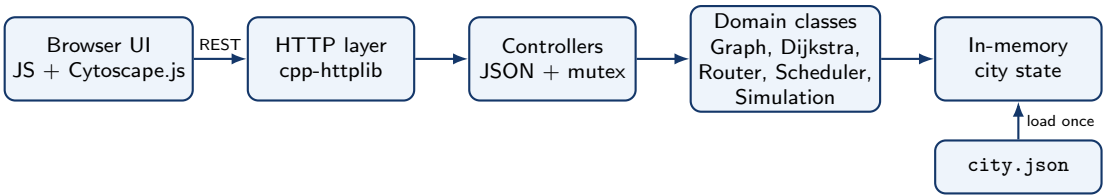
Problem Statement

Garbage trucks follow fixed routes every day whether or not there is anything to collect. Every bin is treated as if it fills at the same rate, and hazardous waste is often collected with ordinary waste on the same residential streets.

Finalized Objectives

1. Model the city as a weighted, zone-tagged graph
2. Build shortest-path collection routes for each truck with Dijkstra's algorithm
3. Schedule pickups from each home's fill rate instead of a fixed timetable
4. Send hazardous waste on industrial and arterial roads only, to the right disposal bin

Updated System Architecture / Workflow



Major Components

- One C++17 process serves the API and front end
- In-memory adjacency-list city from `city.json`
- `RouteStrategy` base; Priority / FIFO subclasses
- REST + JSON, 13 endpoints, one error format

Changes Since Phase-I

- Day-by-day simulation with a limited fleet
- En-route pickup in the route builders

System Overview

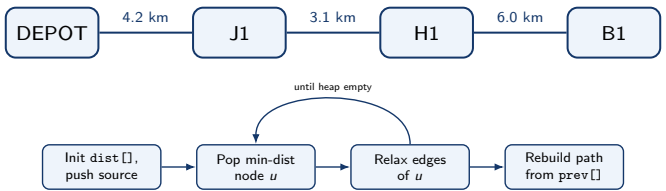
1. The clock moves forward one day.
2. A home is due once the days since its last collection reach capacity / fill rate (1–14 days); the most overdue comes first (b).
3. Due bins are emptied, hazardous homes go to the hazard trucks (e), and normal homes are ordered and packed onto trucks before routing (d).
4. Each truck chains shortest paths in its list order:

```
current = DEPOT;  visited = { }
for each home in order:
    skip it if in visited
    path = Dijkstra(current → home);  if none, mark unreachable
    empty later stops on the path, add them to visited                    (en-route pickup, (c))
    empty home, add to visited;  current = home
then visit the nearest bin for each waste type collected
```

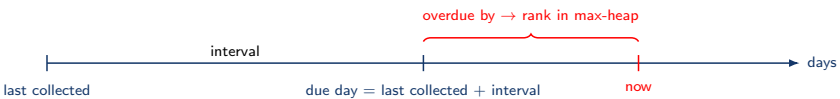
5. Served homes get "last collected = today"; unreachable ones are reported as failed, and carried-over homes rank higher tomorrow.

Graph Storage, Dijkstra & Fill-Rate Scheduling

(a) Graph Storage and Dijkstra

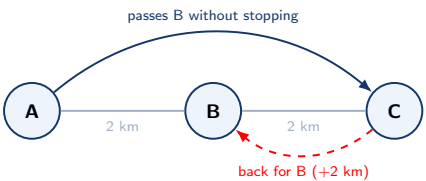


(b) Fill-Rate Scheduling

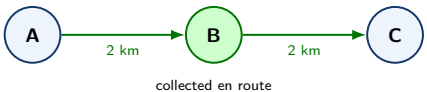


(c) En-Route Pickup

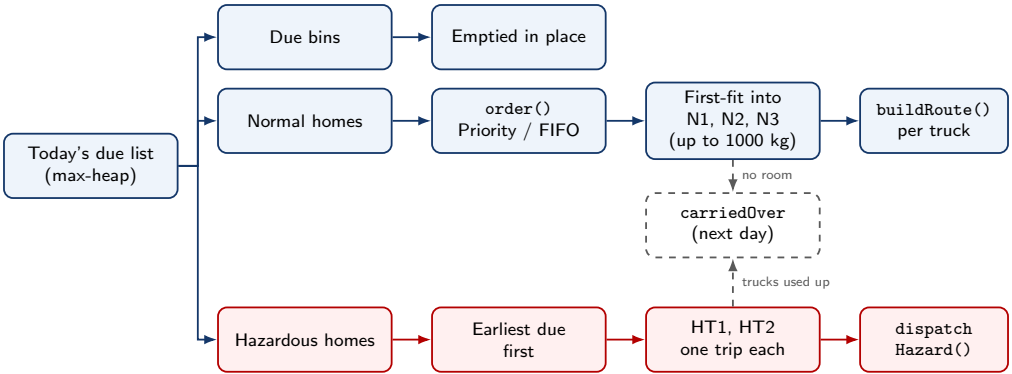
Literal order A → C → B: 6 km



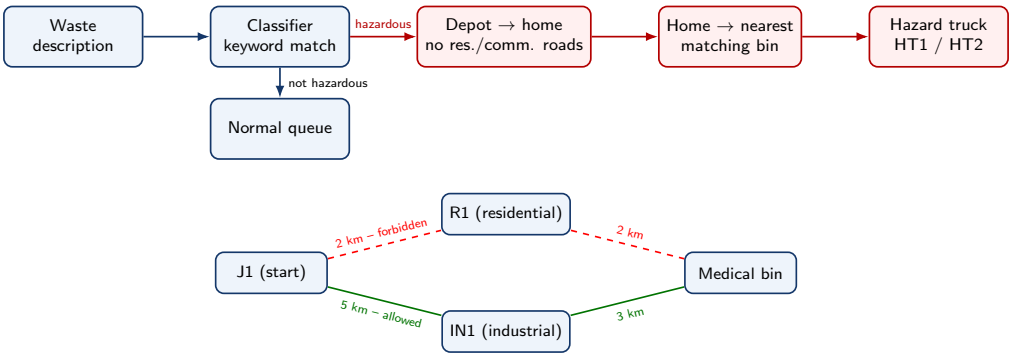
EcoGraph: A → B → C: 4 km



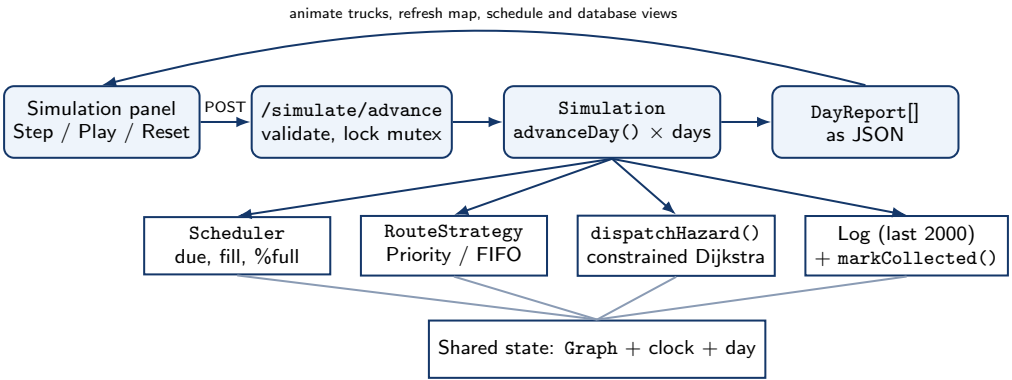
(d) Truck Fleet and Capacity



(e) Hazard Dispatch & Constrained Dijkstra



(f) Simulation Architecture



Module Design & Functional Requirements

Major Modules

1. `graph.cpp`
2. `dijkstra.cpp`
3. `scheduler.cpp`
4. `waste_classifier.cpp`
5. `router.cpp`
6. `simulation.cpp`
7. `controllers.cpp`
8. `graph-view.js`

Functional Requirements

1. City graph as a zone-tagged adjacency list
2. Plain and zone-constrained shortest paths
3. Due, overdue, % full; max-heap of due homes
4. Classifies waste: medical, chemical, recyclable, compost
5. Priority / FIFO routes, en-route pickup, hazard dispatch
6. Advances days, loads trucks, keeps the collection log
7. 13 REST endpoints as JSON; mutex on shared state
8. Map view and truck route animation

Module Dependency

`simulation.cpp` asks `scheduler.cpp` what is due; `router.cpp` builds routes on `dijkstra.cpp`; `controllers.cpp` exposes it all.

Technology Stack & Development Methodology

Technology Stack

- Programming: C++17 (backend), JavaScript (front end)
- Database: in-memory graph loaded from `city.json`
- Framework: cpp-httplib, nlohmann/json, Cytoscape.js
- Development Tool: GCC 13, CMake / `build.sh`
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Implementation of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Graph (adjacency list), min-heap and max-heap (priority queues), hash map (`unordered_map`), path reconstruction via a predecessor map	Graph model, routing engine, scheduler
OOPs with C++	**Encapsulation** (`Graph` hides its node/adjacency maps), **Abstraction** (`Scheduler` / `Classifier` expose `getDueLocations()`, `classifyWaste()`), **Inheritance** (`RouteStrategy` base), **Polymorphism** (`PriorityRouteBuilder` / `FIFORouteBuilder` override virtual `order()`)	Domain-layer C++ classes

Requirement: The DSA structures come together as a set of C++ classes that put all four OOP pillars to use – tying Dijkstra, routing and scheduling into one working backend.

Development & Implementation Progress

Module	Status	Work Completed
Graph + Dijkstra	Completed	Zone-tagged adjacency list, plain and constrained Dijkstra
Scheduler + Classifier	Completed	Fill-rate intervals, due list in a max-heap, keyword waste classifier
Router + Hazard Dispatch	Completed	Priority / FIFO routes, en-route pickup, hazard trucks on safe roads
Simulation + Front End	Completed	Day-by-day simulation with limited fleet; map, schedule and database views
Deployment	Pending	Docker image and hosting

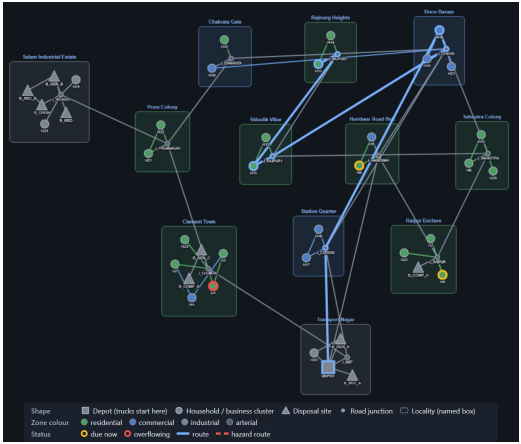
Overall Project Completion: 80%

Working Prototype / Demonstration

Implemented Features

- City modelled as a weighted, zone-tagged graph
- Shortest-path collection route for each truck with Dijkstra's algorithm
- Pickups scheduled from each home's fill rate
- Hazardous waste routed on industrial and arterial roads only, to the right disposal bin

Working Prototype – Map View



Working Prototype – Day-by-Day Summary

Day-by-day summary

Day	Date	Overflowing (no pickups)	Overflowing (EcoRouting)	Homes served	Distance (km)
1	2026-10-18	8	8	8	126.1
2	2026-10-19	15	6	7	131.0
3	2026-10-20	21	3	4	60.5
4	2026-10-21	26	4	5	76.1
5	2026-10-22	28	2	7	167.8
6	2026-10-23	32	0	5	58.4
7	2026-10-24	32	0	7	99.1
8	2026-10-25	33	0	2	41.3
9	2026-10-26	35	0	8	123.4
10	2026-10-27	37	0	7	103.5

Working Prototype – Database View: Nodes and % Full

Fill rate (kg/day)	Capacity (kg)	Fill level (kg)	% full	Last collected	Days since	Interval (d)	Next due	Overdue (d)	Status	Connected to
25.0	350	325.0	93%	2026-10-14	13.0	14.0	2026-10-28	-1.0	ok	J_SELAQUI, B_REC_B
180.0	1440	0.0	0%	2026-10-27	0.0	8.0	2026-11-04	-8.0	ok	J_RAIPUR
120.0	1200	240.0	20%	2026-10-25	2.0	10.0	2026-11-04	-8.0	ok	B_GEN_C
250.0	2500	2000.0	80%	2026-10-19	8.0	10.0	2026-10-29	-2.0	ok	DEPOT, H23
200.0	2400	200.0	8%	2026-10-26	1.0	12.0	2026-11-07	-11.0	ok	J_SELAQUI, B_REC_B, B_MED
160.0	1920	1440.0	75%	2026-10-18	9.0	12.0	2026-10-30	-3.0	ok	J_CLEMENT, B_COMP_B
30.0	390	30.0	8%	2026-10-26	1.0	13.0	2026-11-08	-12.0	ok	B_GEN_B
150.0	1950	300.0	15%	2026-10-25	2.0	13.0	2026-11-07	-11.0	ok	DEPOT
140.0	1960	700.0	36%	2026-10-22	5.0	14.0	2026-11-05	-9.0	ok	B_GEN_B, B_CHEM
									n/a	J_STATION, J_ISBT, J_HARIDWAR, H23, B_GEN_A, B_REC_A
70.0	350	210.0	60%	2026-10-24	3.0	5.0	2026-10-29	-2.0	ok	J_CLEMENT, H3
60.0	300	60.0	20%	2026-10-26	1.0	5.0	2026-10-31	-4.0	ok	J_CLEMENT, H4

Working Prototype – Database View: Localities and Zones

Locality	Nodes	Homes + bins	Due	Overflowing	Avg % full	Total fill rate (kg/day)
Chakrata Gate	3	2	0	0	7%	125.0
Clement Town	8	7	0	0	30%	605.0
Doon Bazaar	4	3	0	0	28%	205.0
Haridwar Road Belt	3	2	0	0	54%	135.0
Prem Colony	3	2	0	0	46%	100.0
Raipur Enclave	5	4	0	0	18%	295.0
Rajmarg Heights	3	2	0	0	61%	105.0
Sahastra Colony	4	3	0	0	21%	135.0
Selam Industrial Estate	7	6	0	0	38%	480.0
Shivalik Vihar	3	2	0	0	50%	135.0
Station Quarter	3	2	0	0	50%	165.0
Transport Nagar	5	3	0	0	58%	460.0

Zones

Zone	Nodes	Homes + bins	Due	Overflowing	Avg % full	Total fill rate (kg/day)
arterial	12	0	0	0	0%	0.0
commercial	8	8	0	0	32%	610.0
industrial	13	12	0	0	42%	1400.0
residential	18	18	0	0	34%	935.0

Working Prototype – Database View: Collection Log

Collection log (newest first)

Day	Date	Location	Type	Action	Truck	Overdue (d)	% full when collected
10	2026-10-27	B_COMP_A	bin	emptied	-	0.0	100%
10	2026-10-27	H9	home	route	N3	0.0	100%
10	2026-10-27	H8	home	route	N2	0.0	100%
10	2026-10-27	H7	home	route	N2	0.0	100%
10	2026-10-27	H4	home	route	N2	0.0	100%
10	2026-10-27	H24	home	route	N1	0.0	100%
10	2026-10-27	H19	home	route	N1	0.0	100%
10	2026-10-27	H15	home	route	N1	0.0	100%
9	2026-10-26	B_GEN_B	bin	emptied	-	0.0	100%
9	2026-10-26	B_MED	bin	emptied	-	0.0	100%
9	2026-10-26	HZ3	home	hazard	HT1	0.0	100%
9	2026-10-26	H3	home	route	N3	0.0	100%

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Dijkstra	Matches hand-solved graphs; no forbidden road used	All cases match	Pass
Scheduler / Router	Intervals in 1–14 days; correct Priority / FIFO order	All cases match	Pass
Simulation	Capacity respected; overflow carried to next day	All cases match	Pass
REST API	Correct JSON; clean error on bad input	Checked with curl and the browser	Pass

Results / Validation

Unit test programs for Dijkstra, the domain classes and the simulation, plus a check of the city data, all pass. In the A, C, B example, en-route pickup cuts the route from 6 km to 4 km.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: `github.com/Space-Fighter/EcoGraph`
- README.md: Updated, with full API reference
- Contributors: Tejas Jain, Apoorv Negi
- Repository Status: Updated
- Branches: `main`

Development Activity

- Regular commits maintained
- Meaningful commit messages
- Individual contribution visible
- Latest implementation uploaded
- Documentation maintained

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Tejas Jain	Team Lead, Backend	Graph, Dijkstra, scheduler, classifier, route builders, hazard dispatch, simulation, REST server, unit tests	17
Apoorv Negi	Front End, Docs	README and API docs, map view, simulation, unit tests, report and presentation	3
Yash Nigam	Front End	Schedule, simulation and database views, map view, unit tests, report and presentation	0

Individual Evaluation

Every member should be prepared to explain and demonstrate their own technical contribution to the project.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Hazard truck had no legal roads with only residential / commercial tags
- Trucks drove past houses on their own list and came back
- With unlimited trucks, scheduling never showed its effect

Solutions / Learning

- Added arterial and industrial road classes; en-route pickup
- Limited fleet (1000 kg per truck) with carry-over to the next day

Phase-III Roadmap

1. Multi-stage Dockerfile for the server and front end
2. Deploy on a container host with auto-redeploy from GitHub
3. Front-end improvements
4. Code cleanup
5. Phase-III report and live demo

Expected Outcomes & References

Expected Final Outcomes

- Working C++ routing and scheduling server with a REST API
- Shortest-path routes per truck, with en-route pickup
- Hazardous waste kept off residential and commercial roads
- Hosted demo reachable from any browser

References

1. E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," 1959
2. Ghiani et al., "Operations research in solid waste management," *Comput. Oper. Res.*, 2014
3. Cormen et al., *Introduction to Algorithms*, 3rd ed., MIT Press
4. cpp-httplib, nlohmann/json and Cytoscape.js documentation

Thank You
Questions & Suggestions